

# Cryptography & Network Security

## MSc Final Exam — Master Study Notes

Compiled from all lectures, PDFs, and class topics | March 2026

01

Contents

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Chapter 1: Network Security Fundamentals</b>     | <b>4</b> |
| 1.1      | Key Definitions . . . . .                           | 4        |
| 1.2      | CIA Security Goals . . . . .                        | 4        |
| 1.3      | Passive vs Active Attacks . . . . .                 | 4        |
| 1.4      | Levels of Impact . . . . .                          | 4        |
| 1.5      | X.800 Security Services . . . . .                   | 4        |
| 1.6      | Security Mechanisms (X.800) . . . . .               | 5        |
| 1.7      | Models for Network Security . . . . .               | 5        |
| <b>2</b> | <b>Chapter 2: Classical Encryption Techniques</b>   | <b>5</b> |
| 2.1      | Basic Terminology . . . . .                         | 5        |
| 2.2      | Caesar Cipher . . . . .                             | 5        |
| 2.3      | Playfair Cipher . . . . .                           | 5        |
| 2.4      | Hill Cipher . . . . .                               | 5        |
| 2.5      | Polyalphabetic Cipher (Vigenère) . . . . .          | 6        |
| 2.6      | Autokey Cipher . . . . .                            | 6        |
| 2.7      | One-Time Pad (OTP) . . . . .                        | 6        |
| 2.8      | Transposition Ciphers . . . . .                     | 6        |
| 2.9      | Steganography . . . . .                             | 6        |
| <b>3</b> | <b>Chapter 3: Block Ciphers and DES</b>             | <b>6</b> |
| 3.1      | Block Cipher vs Stream Cipher . . . . .             | 6        |
| 3.2      | Claude Shannon & Feistel Structure . . . . .        | 6        |
| 3.3      | DES (Data Encryption Standard) . . . . .            | 7        |
| 3.4      | Meet-in-the-Middle Attack & Triple DES . . . . .    | 7        |
| 3.5      | AES (Advanced Encryption Standard) . . . . .        | 7        |
| 3.6      | Cipher Block Modes of Operation . . . . .           | 7        |
| <b>4</b> | <b>Chapter 7: PRNG &amp; Stream Ciphers</b>         | <b>8</b> |
| 4.1      | Traffic Padding . . . . .                           | 8        |
| 4.2      | Pseudorandom Number Generators (PRNG) . . . . .     | 8        |
| <b>5</b> | <b>Chapter 8: Number Theory Foundations</b>         | <b>8</b> |
| 5.1      | Prime Numbers & Factorization . . . . .             | 8        |
| 5.2      | GCD & Relatively Prime Numbers . . . . .            | 8        |
| 5.3      | Euler's Totient Function . . . . .                  | 8        |
| 5.4      | Euler's Theorem & Fermat's Theorem . . . . .        | 9        |
| <b>6</b> | <b>Chapter 9: Public-Key Cryptography &amp; RSA</b> | <b>9</b> |
| 6.1      | Symmetric vs Asymmetric Cryptography . . . . .      | 9        |
| 6.2      | RSA Algorithm . . . . .                             | 9        |
| 6.3      | RSA Security . . . . .                              | 10       |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>7 Chapter 10: Diffie-Hellman, ElGamal, Knapsack</b>           | <b>10</b> |
| 7.1 Diffie-Hellman Key Exchange . . . . .                        | 10        |
| 7.1.1 Man-in-the-Middle Attack on D-H . . . . .                  | 10        |
| 7.2 ElGamal Cryptosystem . . . . .                               | 10        |
| 7.3 Knapsack Encryption (Merkle-Hellman) . . . . .               | 11        |
| <b>8 Chapter 11: Hash Functions &amp; Message Authentication</b> | <b>11</b> |
| 8.1 Hash Functions . . . . .                                     | 12        |
| 8.2 SHA Standard . . . . .                                       | 12        |
| 8.3 Message Authentication Using Hash Functions . . . . .        | 12        |
| 8.4 Digital Signatures with Hash . . . . .                       | 12        |
| <b>9 Chapter 13: Digital Signature Standard (DSS/DSA)</b>        | <b>12</b> |
| <b>10 Chapter 14 &amp; 15: Key Management, X.509, Kerberos</b>   | <b>13</b> |
| 10.1 Key Hierarchy . . . . .                                     | 13        |
| 10.2 Public Key Distribution Methods . . . . .                   | 13        |
| 10.3 Kerberos v4 Protocol . . . . .                              | 13        |
| 10.4 Kerberos Realm . . . . .                                    | 13        |
| <b>11 IP Security (IPSec)</b>                                    | <b>13</b> |
| 11.1 What is IPSec? . . . . .                                    | 13        |
| 11.2 IPSec Protocols . . . . .                                   | 14        |
| 11.3 IPSec Modes . . . . .                                       | 14        |
| <b>12 Quick Reference: All Key Formulas</b>                      | <b>14</b> |
| <b>13 Number Theory Foundations (Lecture 4)</b>                  | <b>17</b> |
| 13.1 Traffic Padding & Random Numbers . . . . .                  | 17        |
| 13.2 Pseudorandom Number Generator (PRNG) . . . . .              | 17        |
| 13.2.1 Linear Congruential Generator (LCG) . . . . .             | 17        |
| 13.3 Prime Numbers & Prime Factorization . . . . .               | 17        |
| 13.4 Relatively Prime Numbers & GCD . . . . .                    | 18        |
| 13.5 Euler's Totient Function $\phi(n)$ . . . . .                | 18        |
| 13.6 Euler's Theorem . . . . .                                   | 18        |
| <b>14 Chapter 9: Public-Key Cryptography &amp; RSA</b>           | <b>19</b> |
| 14.1 Private-Key vs. Public-Key Cryptography . . . . .           | 19        |
| 14.2 RSA Algorithm . . . . .                                     | 19        |
| 14.2.1 RSA Key Generation Steps . . . . .                        | 20        |
| 14.2.2 Encryption and Decryption . . . . .                       | 20        |
| 14.2.3 RSA Algorithm Flow . . . . .                              | 20        |
| 14.2.4 RSA Full Example . . . . .                                | 21        |
| 14.3 RSA Security . . . . .                                      | 21        |
| <b>15 Chapter 10: Diffie-Hellman Key Exchange &amp; Attacks</b>  | <b>21</b> |
| 15.1 Diffie-Hellman Key Exchange . . . . .                       | 21        |
| 15.1.1 Global Public Parameters (DH Setup) . . . . .             | 21        |
| 15.1.2 DH Protocol Steps . . . . .                               | 22        |
| 15.2 Man-in-the-Middle (MITM) Attack on DH . . . . .             | 22        |
| 15.3 Meet-in-the-Middle (MITM) Attack . . . . .                  | 22        |
| 15.3.1 The Problem with Double Encryption . . . . .              | 22        |
| 15.3.2 Algorithm Steps . . . . .                                 | 23        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>16 ElGamal Cryptosystem</b>                         | <b>23</b> |
| 16.1 Key Generation . . . . .                          | 23        |
| 16.2 Encryption . . . . .                              | 23        |
| 16.3 Decryption . . . . .                              | 24        |
| 16.4 ElGamal Example . . . . .                         | 24        |
| 16.5 Advantages & Disadvantages . . . . .              | 24        |
| 16.6 Short Note: ElGamal . . . . .                     | 25        |
| <b>17 Knapsack Encryption (Merkle–Hellman)</b>         | <b>25</b> |
| 17.1 Overview . . . . .                                | 25        |
| 17.2 Super-Increasing Sequence (Private Key) . . . . . | 25        |
| 17.3 Deriving the Public Key . . . . .                 | 25        |
| 17.4 Encryption . . . . .                              | 26        |
| 17.5 Decryption . . . . .                              | 26        |

## 1: Network Security Fundamentals

### 1.1 Key Definitions

#### Definitions

- **Computer Security:** Protecting data/systems; preserving integrity, availability, confidentiality of hardware, software, data.
- **Network Security:** Measures to protect data *during transmission*.
- **Internet Security:** Protecting data during transmission over interconnected networks.

### 1.2 CIA Security Goals

- **Confidentiality:** Hidden from unauthorized access. Threatened by: Snooping, Traffic analysis.
- **Integrity:** Protected from unauthorized change. Threatened by: Modification, Masquerading, Replay, Repudiation.
- **Availability:** Available to authorized entity when needed. Threatened by: Denial of Service.

### 1.3 Passive vs Active Attacks

**Passive** (difficult to detect, no data alteration):

- Release of message contents — opponent learns content.
- Traffic analysis — determines communication patterns even if encrypted.

**Active** (difficult to prevent, but detectable):

- **Masquerade:** One entity pretends to be another.
- **Replay:** Capture and retransmit to produce unauthorized effect.
- **Modification:** Alter, delay, or reorder a message.
- **Denial of Service (DoS):** Prevents normal use of resources.

Four asset-attack categories: **Interruption** (availability), **Interception** (confidentiality), **Modification** (integrity), **Fabrication** (authenticity).

### 1.4 Levels of Impact

| Level    | Description                                                                             |
|----------|-----------------------------------------------------------------------------------------|
| Low      | Limited adverse effect; minor damage; functions still performable.                      |
| Moderate | Serious adverse effect; significant damage, financial loss, or harm.                    |
| High     | Severe/catastrophic; loss of primary functions; major financial loss; life-threatening. |

### 1.5 X.800 Security Services

- **Authentication:** Communicating entity is genuine.
- **Access Control:** Prevention of unauthorized resource use.
- **Data Confidentiality:** Protection from unauthorized disclosure.
- **Data Integrity:** Data received exactly as sent by authorized entity.
- **Non-Repudiation:** Protection against denial by sender/receiver.

## 1.6 1Security Mechanisms (X.800)

Encipherment, Digital Signature, Access Control, Data Integrity, Authentication Exchange, Traffic Padding, Routing Control, Notarization.

## 1.7 1Models for Network Security

- **Network Security Model:** Sender → Security Transform (key) → Untrusted Channel → Receiver. Requires: algorithm design, key generation, key distribution, protocol specification.
- **Network Access Security Model:** Gatekeeper functions to identify users; controls to restrict access; trusted systems.

## 2 1

## Chapter

## 2: Classical Encryption Techniques

### 2.1 1Basic Terminology

#### Core Terms

- **Plaintext:** Original message.
- **Ciphertext:** Encrypted message.
- **Cipher:** Algorithm for transforming plaintext to ciphertext.
- **Key:** Secret info used by cipher known only to sender/receiver.
- **Cryptanalysis:** Study of breaking ciphertext without knowing the key.
- **Cryptology:** Cryptography + Cryptanalysis.

Classification: (1) Type of operations (substitution/transposition); (2) Number of keys (symmetric/asymmetric); (3) Processing method (block/stream).

### 2.2 1Caesar Cipher

#### Caesar Cipher

$$C = E(p) = (p + k) \bmod 26 \quad P = D(c) = (c - k) \bmod 26$$

Example ( $k = 3$ ): meet me → PHHW PH

### 2.3 1Playfair Cipher

- Uses  $5 \times 5$  matrix built from a keyword (I/J = one letter). Encrypts **digrams**.
- **Rule 1:** Repeated letters in pair separated by filler 'x'.
- **Rule 2:** Same row → letter to the right (wrap around).
- **Rule 3:** Same column → letter below (wrap around).
- **Rule 4:** Otherwise → letter at same row, other letter's column.
- Strength: 676 digrams possible; harder than monoalphabetic.

### 2.4 1Hill Cipher

#### Hill Cipher

Takes  $m$  plaintext letters; key =  $m \times m$  invertible matrix  $K$ .

$$\mathbf{C} = K\mathbf{P} \bmod 26 \quad \mathbf{P} = K^{-1}\mathbf{C} \bmod 26$$

Example:  $3 \times 3$  key matrix, plaintext DEF → vector  $[3 \ 4 \ 5]$ , multiply by  $K \bmod 26$ .

## 2.5 1Polyalphabetic Cipher (Vigenère)

- Multiple Caesar ciphers; key  $K = k_1 k_2 \dots k_d$  (repeating).
- $i$ -th letter of plaintext shifted by  $k_i$ .
- Example: Plaintext THISPROCESS, Key CIPHER  $\rightarrow$  VPXZTIQK...

## 2.6 1Autokey Cipher

- Key = initial keyword + appended plaintext itself.
- More secure than Vigenère; no simple repeating key.
- Example: PT MEETMEFORLUNCH, Key RESTAURANTMEET  $\rightarrow$  DIWMMYWOOEGRGA.

## 2.7 1One-Time Pad (OTP)

### One-Time Pad — Perfect Secrecy

Key is truly random, same length as plaintext, used **exactly once**.

$$C = M \oplus \text{OTP} \quad M = C \oplus \text{OTP}$$

Provides **perfect secrecy** (Shannon). Impractical at scale (key as long as message).

## 2.8 1Transposition Ciphers

Rearrange (not replace) letters.

- **Rail Fence**: Write diagonally across  $n$  rails; read row by row.
- **Row/Column Transposition**: Write in rows; reorder columns by key; read columns.
- **Vernam Cipher**: XOR-based stream cipher.

## 2.9 1Steganography

Conceals *existence* of message (vs. cryptography that conceals content).

Techniques: character marking, invisible ink, pin punctures.

**Drawbacks**: Large overhead to hide few bits; once system discovered, worthless.

## 3 1

## Chapter

### 3: Block Ciphers and DES

#### 3.1 1Block Cipher vs Stream Cipher

| Feature         | Block Cipher          | Stream Cipher         |
|-----------------|-----------------------|-----------------------|
| Processing unit | Block (64/128 bits)   | Bit or byte at a time |
| Design          | Simpler               | More complex          |
| Operations      | Confusion + Diffusion | Confusion (XOR) only  |
| Reversibility   | Hard to reverse       | Easy (XOR invertible) |
| Examples        | DES, AES              | RC4, A5/1             |

#### 3.2 1Claude Shannon & Feistel Structure

- **Confusion**: Makes relationship between ciphertext and key as complex as possible (via substitution/S-boxes).
- **Diffusion**: Dissipates statistical structure of plaintext over ciphertext (via permutation/P-boxes).

**Feistel Round Function**

Block split into halves  $L_0, R_0$ . Each round:

$$L_i = R_{i-1} \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Decryption: same algorithm, subkeys in **reverse** order.

**3.3 1DES (Data Encryption Standard)**

- Block size: **64 bits**; Key: **56 bits** effective (+ 8 parity = 64 bits total).
- **16 Feistel rounds**.
- Each round:  $R$  expanded 32  $\rightarrow$  48 bits, XOR with 48-bit subkey, S-box substitution 48  $\rightarrow$  32 bits, P-box permutation.
- **Key Scheduling**: 56-bit key  $\rightarrow$  two 28-bit halves; shifted and permuted to produce 16 subkeys of 48 bits each.
- **Decryption**: Identical algorithm; subkeys in reverse order.
- **Avalanche Effect**: 1-bit change in plaintext/key causes many ciphertext bits to change. Highly desirable.
- **Weakness**: 56-bit key is too short; vulnerable to brute-force. Replaced by AES.

**3.4 1Meet-in-the-Middle Attack & Triple DES****Meet-in-the-Middle Attack on Double DES**

Double DES:  $C = E_{K_2}(E_{K_1}(P))$ . Intermediate:  $X = E_{K_1}[P] = D_{K_2}[C]$ .

**Attack**: Encrypt  $P$  with all  $2^{56}$  keys; store. Decrypt  $C$  with all  $2^{56}$  keys; match  $X$ .

Cost:  $O(2^{56})$ , **not**  $O(2^{112}) \Rightarrow$  Double DES is **insecure**!

**3DES Two-Key** (E-D-E):  $C = E_{K_1}[D_{K_2}[E_{K_1}[P]]]$  (standardized in ANSI X9.17)

**3DES Three-Key**:  $C = E_{K_3}[D_{K_2}[E_{K_1}[P]]]$  (strongest; used in Internet apps)

**3.5 1AES (Advanced Encryption Standard)**

- Proposed to succeed DES. Block size: **128 bits**. Key sizes: 128, 192, or 256 bits.
- **NOT** Feistel — processes entire block in parallel.
- 128-bit key: expands to  $44 \times 32$ -bit words; 4 words per round. 10/12/14 rounds.

**3.6 1Cipher Block Modes of Operation**

| Mode | Formula                                       | Key Points                                                                                |
|------|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| ECB  | $C_i = E_K(P_i)$                              | Blocks independent; same $P \rightarrow$ same $C$ ; vulnerable to pattern/replay attacks. |
| CBC  | $C_i = E_K(P_i \oplus C_{i-1})$               | Chained; needs IV; gold standard for bulk encryption.                                     |
| CFB  | $C_i = P_i \oplus E_K(C_{i-1})$               | Stream mode; for byte-at-a-time data.                                                     |
| OFB  | $C_i = P_i \oplus O_i$ ; $O_i = E_K(O_{i-1})$ | Feedback independent of message; good for noisy channels.                                 |
| CTR  | $C_i = P_i \oplus E_K(\text{Nonce} + i)$      | Parallel processing; no padding; unique nonce required.                                   |

## 7: PRNG &amp; Stream Ciphers

## 4.1 1Traffic Padding

Produces ciphertext *continuously*, even without plaintext. Random data encrypted when no real plaintext; attacker cannot deduce real traffic volume.

## 4.2 1Pseudorandom Number Generators (PRNG)

- **True random:** Statistically random, unpredictable — impractical in bulk.
- **Pseudorandom:** Deterministic algorithm; passes randomness tests; practical.
- Uses: Nonces, session keys, public-key generation, one-time pad keystream.

## Linear Congruential Generator (LCG)

$$X_{n+1} = (aX_n + c) \bmod m$$

Criteria: full-period, appears random, efficient with 32-bit arithmetic.

**Weakness:** Attacker can reconstruct sequence from a few values.

## ANSI X9.17 PRNG (Strongest Cryptographic PRNG)

Uses date/time  $DT_i$ , seed  $V_i$ , and Triple-DES EDE encryptions:

$$R_i = \text{EDE}([K1, K2], V_i \oplus \text{EDE}([K1, K2], DT_i))$$

$$V_{i+1} = \text{EDE}([K1, K2], R_i \oplus \text{EDE}([K1, K2], DT_i))$$

Uses 112-bit key; 9 DES operations per cycle; two independent pseudorandom inputs.

## 8: Number Theory Foundations

## 5.1 1Prime Numbers &amp; Factorization

- Primes have divisors of 1 and themselves only. E.g.: 2, 3, 5, 7, 11, 13, 17, 19...
- Prime factorization:  $91 = 7 \times 13$ ;  $3600 = 2^4 \times 3^2 \times 5^2$ .
- Factoring a large number is hard; multiplying factors together is easy (asymmetry used in RSA).

## 5.2 1GCD &amp; Relatively Prime Numbers

$a, b$  are **relatively prime** if  $\gcd(a, b) = 1$ .

Example:  $\gcd(8, 15) = 1$  (relatively prime);  $\gcd(300, 18) = 6$ .

## 5.3 1Euler's Totient Function

Euler's Totient  $\phi(n)$ 

Count of integers in  $[1, n]$  relatively prime to  $n$ .

$$\phi(p) = p - 1 \quad (p \text{ prime}) \quad \phi(pq) = (p - 1)(q - 1) \quad (p, q \text{ distinct primes})$$

Examples:  $\phi(37) = 36$ ;  $\phi(21) = (3 - 1)(7 - 1) = 12$ ;  $\phi(10) = |\{1, 3, 7, 9\}| = 4$ .



## 5.4 1Euler's Theorem & Fermat's Theorem

### Euler's Theorem

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad \text{for } \gcd(a, n) = 1$$

Example:  $a = 3, n = 10 \Rightarrow 3^4 = 81 \equiv 1 \pmod{10}$ .

**Fermat's Little Theorem** (special case,  $n = p$  prime):  $a^{p-1} \equiv 1 \pmod{p}$ .

## 6 1

## Chapter

## 9: Public-Key Cryptography & RSA

### 6.1 1Symmetric vs Asymmetric Cryptography

| Symmetric (Private Key)                   | Asymmetric (Public Key)                        |
|-------------------------------------------|------------------------------------------------|
| One shared key                            | Two keys: public + private                     |
| Symmetric; parties equal                  | Asymmetric; parties not equal                  |
| Key compromise = all messages compromised | Private key stays secret; public freely shared |
| Fast; for bulk data                       | Slow; for key exchange & signatures            |

Invented by Diffie & Hellman at Stanford, 1976. Solves: (1) key distribution without trusted channel; (2) digital signatures.

### 6.2 1RSA Algorithm

By Rivest, Shamir, Adleman (MIT, 1977). Based on difficulty of **factoring large integers**.

#### RSA Key Generation

**Step 1:** Select two large primes  $p \neq q$ .

**Step 2:** Compute  $n = p \times q$ .

**Step 3:** Compute  $\phi(n) = (p-1)(q-1)$ .

**Step 4:** Select  $e$ :  $\gcd(e, \phi(n)) = 1$  and  $1 < e < \phi(n)$ .

**Step 5:** Compute  $d = e^{-1} \pmod{\phi(n)}$ , i.e.,  $ed \equiv 1 \pmod{\phi(n)}$ .

**Step 6:** **Public key**  $KU = \{e, n\}$ ; **Private key**  $KR = \{d, p, q\}$ .

#### RSA Encryption & Decryption

**Encryption:**  $C = M^e \pmod{n} \quad (M < n)$

**Decryption:**  $M = C^d \pmod{n}$

**Why it works (Euler):**  $C^d = M^{ed} = M^{1+k\phi(n)} = M \cdot (M^{\phi(n)})^k \equiv M \pmod{n}$

#### RSA Worked Example 1

$p = 17, q = 11 \Rightarrow n = 187, \phi(n) = 160$ .

Choose  $e = 7$  ( $\gcd(7, 160) = 1$ ). Find  $d$ :  $7d \equiv 1 \pmod{160} \Rightarrow d = 23$  (since  $7 \times 23 = 161$ ).

**Public:**  $\{7, 187\}$ ; **Private:**  $\{23, 17, 11\}$ .

Encrypt  $M = 88$ :  $C = 88^7 \pmod{187} = 11$ .

Decrypt  $C = 11$ :  $M = 11^{23} \pmod{187} = 88 \checkmark$

**RSA Worked Example 2**

$p = 13, q = 11 \Rightarrow n = 143, \phi(n) = 120$ .

Choose  $e = 13$  ( $\gcd(13, 120) = 1$ ). Find  $d$ :  $d = \frac{120i+1}{13}$ ;  $i = 4 \Rightarrow d = 37$ .

**Public:**  $\{13, 143\}$ ; **Private:**  $\{37, 143\}$ .

Encrypt  $M = 13$ :  $C = 13^{13} \bmod 143 = 52$ .

Decrypt  $C = 52$ :  $M = 52^{37} \bmod 143 = 13 \checkmark$

**6.3 1RSA Security**

- **Brute force:** Infeasible (1024-bit key).
- **Mathematical attack:** Factor  $n$  to get  $p, q$  and compute  $\phi(n)$ ; computationally infeasible for large  $n$ .
- **Timing attacks:** Exploit decryption time variations.

**7 1****Chapter****10: Diffie-Hellman, ElGamal, Knapsack****7.1 1Diffie-Hellman Key Exchange**

Key distribution only (cannot encrypt messages). Security: difficulty of computing **discrete logarithms**.

**Diffie-Hellman Setup & Exchange**

**Global public params:** Large prime  $q$ ; primitive root  $\alpha$  of  $q$ .

**Alice:** secret  $x_A < q$ ; public  $y_A = \alpha^{x_A} \bmod q$ .

**Bob:** secret  $x_B < q$ ; public  $y_B = \alpha^{x_B} \bmod q$ .

**Shared key:**  $K = y_B^{x_A} \bmod q = y_A^{x_B} \bmod q = \alpha^{x_A x_B} \bmod q$

**Diffie-Hellman Example**

$q = 353, \alpha = 3$ .

Alice:  $x_A = 97, y_A = 3^{97} \bmod 353 = 40$ .

Bob:  $x_B = 233, y_B = 3^{233} \bmod 353 = 248$ .

Shared key:  $K = 248^{97} \bmod 353 = 40^{233} \bmod 353 = \mathbf{160} \checkmark$

**1Man-in-the-Middle Attack on D-H**

Darth intercepts  $y_A$  and  $y_B$ ; substitutes his own public keys; independently establishes separate shared keys with Alice and Bob. Darth decrypts, reads/modifies, re-encrypts all messages. **D-H has no authentication** — vulnerable.

**Steps:**

1. Darth creates two key pairs.
2. Alice sends  $y_A$  to Bob; Darth intercepts; sends his own  $y_{D1}$  to Bob.
3. Bob computes  $K_{BD}$  (shared with Darth, not Alice). Bob sends  $y_B$ ; Darth intercepts; sends  $y_{D2}$  to Alice.
4. Alice computes  $K_{AD}$  (shared with Darth, not Bob).
5. Darth intercepts all traffic; decrypts with one key; re-encrypts with the other.

**7.2 1ElGamal Cryptosystem**

Public-key system related to D-H. Security: discrete logarithm problem. Ciphertext is **twice the size** of plaintext.

**ElGamal Key Generation**

Choose prime  $q$ , primitive root  $\alpha$ . User A: choose secret  $x_A$  ( $1 < x_A < q - 1$ ); compute public  $y_A = \alpha^{x_A} \bmod q$ .

**Public key:**  $\{q, \alpha, y_A\}$ ; **Private key:**  $\{x_A\}$ .

**ElGamal Encryption (Bob  $\rightarrow$  Alice)**

1. Represent message  $M$ :  $0 \leq M \leq q - 1$ .
2. Choose random  $k$ ,  $1 \leq k \leq q - 1$  (**different every time!**).
3. Compute one-time key:  $K = y_A^k \bmod q$ .
4.  $C_1 = \alpha^k \bmod q$ ;  $C_2 = K \cdot M \bmod q$ .
5. Send  $(C_1, C_2)$ .

**ElGamal Decryption (Alice recovers  $M$ )**

1. Recover key:  $K = C_1^{x_A} \bmod q$ .
2. Compute:  $K^{-1} \bmod q$ .
3. Recover:  $M = C_2 \cdot K^{-1} \bmod q$ .

**ElGamal Complete Example**

$q = 19$ ,  $\alpha = 10$ . Alice:  $x_A = 5$ ;  $y_A = 10^5 \bmod 19 = 3$ .

Bob sends  $M = 17$ , chooses  $k = 6$ :

$K = 3^6 \bmod 19 = 7$ ;  $C_1 = 10^6 \bmod 19 = 11$ ;  $C_2 = 7 \times 17 \bmod 19 = 5$ .

Bob sends  $(11, 5)$ .

Alice:  $K = 11^5 \bmod 19 = 7$ ;  $K^{-1} = 7^{-1} \bmod 19 = 11$ ;  $M = 5 \times 11 \bmod 19 = \mathbf{17} \checkmark$

**Advantages:** No patent; free to use.

**Disadvantages:** Ciphertext double size;  $k$  must be unique each time (reuse = insecure).

**7.3 1Knapsack Encryption (Merkle-Hellman)**

Based on the **subset sum problem** (NP-hard in general). Uses a superincreasing sequence.

**Knapsack Algorithm**

**Private key:** Superincreasing sequence  $\mathbf{a} = (a_1, \dots, a_n)$  where  $a_i > \sum_{j < i} a_j$ ; choose  $m > \sum a_i$ ; choose  $w$  with  $\gcd(w, m) = 1$ .

**Public key:**  $b_i = w \cdot a_i \bmod m$ .

**Encrypt** binary message  $\mathbf{x} = (x_1, \dots, x_n)$ :  $C = \sum_{i=1}^n x_i b_i$ .

**Decrypt:**  $C' = w^{-1} \cdot C \bmod m$ ; then solve superincreasing knapsack for  $x_i$  by greedy (subtract largest  $a_i \leq C'$ ).

**Example:**  $\mathbf{a} = (1, 2, 4, 10, 20)$ ;  $m = 41$ ;  $w = 18$ ;  $w^{-1} = 16$ .

Public key:  $\mathbf{b} = (18, 36, 31, 21, 2)$ .

Encrypt 10110:  $C = 18 + 0 + 31 + 21 + 0 = 70$ .

Decrypt:  $C' = 16 \times 70 \bmod 41 = 35$ ;  $35 = 20 + 10 + 4 + 1 \Rightarrow 10110 \checkmark$

**Security:** Broken by Shamir (1982) using lattice reduction. No longer secure.

## 8.1 1Hash Functions

### Hash Function

$h = H(M)$ : variable-length input  $M \rightarrow$  fixed-size output  $h$ . Hash is public; used to detect changes to message.

#### Six Requirements:

1. Applicable to any size message.
2. Produces fixed-length output.
3.  $H(M)$  easy to compute.
4. **One-way / Pre-image resistance**: Given  $h$ , infeasible to find  $x$  with  $H(x) = h$ .
5. **Weak collision resistance**: Given  $x$ , infeasible to find  $y \neq x$  with  $H(y) = H(x)$ .
6. **Strong collision resistance**: Infeasible to find *any*  $(x, y)$  with  $H(x) = H(y)$ .

**Uses**: Message authentication, digital signatures, password storage (store hash not password), virus/intrusion detection, PRNG.

## 8.2 1SHA Standard

- **SHA-1** (NIST/NSA 1993, revised 1995): 160-bit hash; used with DSA (FIPS 180-1); based on MD4. Security concerns arose in 2005.
- **SHA-2** (FIPS 180-2, 2002): SHA-256 (256-bit), SHA-384, SHA-512. Designed for AES-era security.

## 8.3 1Message Authentication Using Hash Functions

- **Method (a)**: Encrypt  $(M \| H(M))$  with symmetric key. Provides authentication + confidentiality.
- **Method (b)**: Encrypt only  $H(M)$  with symmetric key. Less computation; no message confidentiality.
- **Method (c)**: Compute  $H(M \| S)$  where  $S$  is shared secret (not transmitted); append to  $M$ . No encryption. Fast.
- **Method (d)**: Encrypt  $(M \| H(M \| S))$  with symmetric key. Both authentication and confidentiality.

## 8.4 1Digital Signatures with Hash

- $H(M)$  encrypted with sender's **private key** = digital signature.
- Anyone with sender's public key can verify.
- Provides **authentication** + **non-repudiation**.

## 9 1 13: Digital Signature Standard (DSS/DSA)

## Chapter

US Govt approved scheme; NIST & NSA; FIPS-186 (1991). Uses SHA hash. **DSA** (Digital Signature Algorithm) is the algorithm. Signature only (unlike RSA). Security: discrete logarithms. Smaller & faster than RSA.

### DSA Global Parameters

Choose 160-bit prime  $q$ ; large prime  $p$  ( $L = 512$  to  $1024$  bits) such that  $q | (p - 1)$ ;  $g = h^{(p-1)/q} \bmod p$  ( $h > 1, g > 1$ ).

**Each user**: private key  $x < q$ ; public key  $y = g^x \bmod p$ .

**DSA Signing**

Generate random  $k < q$  (never reuse!).

$$r = (g^k \bmod p) \bmod q$$

$$s = [k^{-1}(H(M) + xr)] \bmod q$$

Send  $(r, s)$  with message  $M$ .

**DSA Verification**

$$w = s^{-1} \bmod q; \quad u_1 = [H(M) \cdot w] \bmod q; \quad u_2 = (r \cdot w) \bmod q$$

$$v = [(g^{u_1} y^{u_2}) \bmod p] \bmod q$$

Valid if  $v = r$ .

## 10 1 14 & 15: Key Management, X.509, Kerberos

## Chapter

**10.1 1Key Hierarchy**

- **Session Key:** Temporary; one logical session; then discarded.
- **Master Key:** Encrypts session keys; shared by user & KDC.

**10.2 1Public Key Distribution Methods**

1. Public announcement (forgeable).
2. Public directory (tamperable).
3. Public-key authority (requires real-time access).
4. **X.509 Certificates** (best): CA signs certificate binding identity to public key; no real-time access needed.

**10.3 1Kerberos v4 Protocol**

Centralized authentication (MIT). Requirements: Secure, Reliable, Transparent, Scalable.

| Phase            | Exchange                                                | Purpose                      |
|------------------|---------------------------------------------------------|------------------------------|
| (a) Auth Service | $C \rightarrow AS: ID_C \  ID_{tgs} \  TS_1$            | Request TGT                  |
|                  | $AS \rightarrow C: E_{K_C}[K_{C,tgs} \  Ticket_{tgs}]$  | Return encrypted TGT         |
| (b) TGS          | $C \rightarrow TGS: ID_V \  Ticket_{tgs} \  Auth_C$     | Request service ticket       |
|                  | $TGS \rightarrow C: E_{K_{C,tgs}}[K_{C,V} \  Ticket_V]$ | Service ticket + session key |
| (c) Service      | $C \rightarrow V: Ticket_V \  Auth_C$                   | Access service               |
|                  | $V \rightarrow C: E_{K_{C,V}}[TS_5 + 1]$                | Mutual authentication        |

$$Ticket_{tgs} = E_{K_{tgs}}[K_{C,tgs} \| ID_C \| AD_C \| ID_{tgs} \| TS_2 \| Lifetime_2]$$

**10.4 1Kerberos Realm**

Full-service environment = Kerberos server + clients + application servers. Inter-realm requires KDCs to share secret key with each other.

## 11 1 Security (IPSec)

## IP

**11.1 1What is IPSec?**

Set of protocols providing security at the **network (IP) layer** — authenticates and encrypts each IP packet. Secures communication over untrusted networks (Internet, VPN).

## 11.2 IPsec Protocols

- **Authentication Header (AH):** Data integrity + origin authentication + replay protection. No confidentiality.
- **Encapsulating Security Payload (ESP):** Confidentiality + integrity + authentication. Can encrypt payload.
- **IKE (Internet Key Exchange):** Key management and security associations.

## 11.3 IPsec Modes

| Mode      | Description                                               | Use                     |
|-----------|-----------------------------------------------------------|-------------------------|
| Transport | Protects payload; original IP header intact.              | Host-to-host            |
| Tunnel    | Entire original packet encapsulated; new IP header added. | VPN, gateway-to-gateway |

## 12 1

## Quick

### Reference: All Key Formulas

#### Master Formula Sheet

**Caesar:**  $C = (p + k) \bmod 26$

**Hill:**  $C = KP \bmod 26$

**OTP:**  $C = M \oplus \text{Key}$

**Feistel:**  $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

**CBC:**  $C_i = E_K(P_i \oplus C_{i-1})$

**LCG:**  $X_{n+1} = (aX_n + c) \bmod m$

**Euler:**  $a^{\phi(n)} \equiv 1 \pmod{n}$

**Totient:**  $\phi(pq) = (p-1)(q-1)$

**RSA keygen:**  $ed \equiv 1 \pmod{\phi(n)}$

**RSA Enc:**  $C = M^e \bmod n$

**RSA Dec:**  $M = C^d \bmod n$

**DH public:**  $y_A = \alpha^{x_A} \bmod q$

**DH shared:**  $K = y_B^{x_A} \bmod q$

**EG Enc:**  $C_1 = \alpha^k \bmod q$ ;  $C_2 = KM \bmod q$

**EG Dec:**  $K = C_1^{x_A} \bmod q$ ;  $M = C_2 K^{-1} \bmod q$

**DSA sign:**  $r = (g^k \bmod p) \bmod q$

**DSA verify:**  $v = (g^{u_1} y^{u_2} \bmod p) \bmod q = r?$

**3DES-2K:**  $C = E_{K1}[D_{K2}[E_{K1}[P]]]$

**3DES-3K:**  $C = E_{K3}[D_{K2}[E_{K1}[P]]]$

#### Top 15 High-Probability Exam Questions

1. RSA algorithm: complete key generation + full numerical example.
2. Meet-in-the-Middle attack on 2DES; why 3DES is needed.
3. Diffie-Hellman key exchange with Man-in-the-Middle attack + example.
4. ElGamal cryptosystem: setup, encrypt, decrypt with full example.
5. Short note on ElGamal: working, advantages, disadvantages.
6. Knapsack encryption: derive public key from superincreasing sequence + example.
7. Feistel cipher structure with confusion and diffusion.
8. DES algorithm: structure, 16 rounds, key scheduling, avalanche effect.
9. PRNG: LCG formula, ANSI X9.17 PRNG construction.
10. IP Security: what is IPsec? Explain AH and ESP; transport vs tunnel mode.
11. Kerberos v4 protocol: all three phases with ticket structure.
12. Block vs stream cipher; cipher modes: ECB, CBC, CFB, OFB, CTR.
13. Passive vs active attacks with types and examples.
14. Euler's totient function with examples; Euler's theorem proof/application.
15. Hash functions: six requirements, SHA, message authentication methods (a-d).

## Confusion and Diffusion

### Confusion

- ★ Making the relationship between the encryption key and the ciphertext as complex as possible.
- ★ Relationship between CT and PT is obscured.
- ★ Given CT, no info about PT, Key, Encryption algorithm etc.,
- ★ Example: Substitution.

### Diffusion

- ★ Making each plaintext bit affect as many ciphertext bits as possible.
- ★ 1 bit change in PT, significant effect on CT.
- ★ Example: Transposition or Permutation.

## Stream Cipher

Each plaintext digit is encrypted one at a time with the corresponding digit of the keystream, to give a digit of the ciphertext stream.

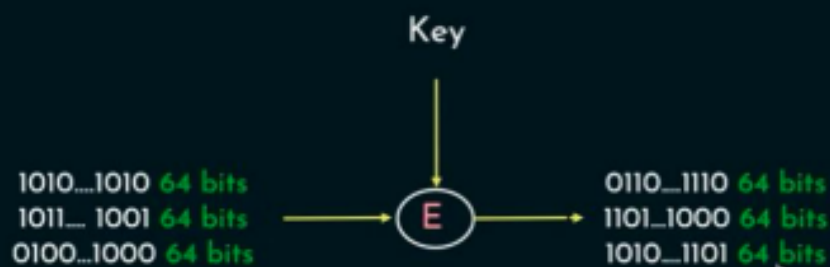


Figure 1: Enter Caption



## Block Cipher

A block cipher is a deterministic algorithm operating on fixed-length groups of bits, called blocks.



## Stream Cipher and Block Cipher

|            | Stream Cipher                                   | Block Cipher                                               |
|------------|-------------------------------------------------|------------------------------------------------------------|
| Length     | Bit or Byte                                     | Block Size - 64 bits, 128 bits                             |
| Design     | Complex                                         | Simple                                                     |
| Principle  | Confusion                                       | Confusion and Diffusion                                    |
| Speed      | Faster                                          | Slower                                                     |
| Encryption | CFB (Cipher Feedback) and OFB (Output Feedback) | Electronic Code Book (ECB) and Cipher Block Chaining (CBC) |
| Decryption | XOR                                             | Reverse of encryption                                      |
| Example    | Vernam Cipher                                   | DES, AES                                                   |

Figure 2: Enter Caption



## 13 1

## Theory Foundations (Lecture 4)

## 13.1 1Traffic Padding &amp; Random Numbers

## Definition 13.1 — Random Number

A **random number** is a value produced by a process whose outcome is unpredictable and statistically independent of previous values.

**Traffic Padding** is a link-layer technique that generates spurious (dummy) traffic to make it difficult for an attacker to analyse communication patterns. It keeps a constant data stream flowing even when no real data is being sent, hiding the timing and volume of actual communications.

## 13.2 1Pseudorandom Number Generator (PRNG)

A **PRNG** is a deterministic algorithm that, given a *seed*, produces a long sequence of values that appear random. PRNGs are crucial for key generation, nonces, and session tokens in cryptographic systems.

## 1Linear Congruential Generator (LCG)

## LCG Recurrence Relation

$$X_{n+1} = (a \cdot X_n + c) \bmod m$$

- $m$  — modulus ( $m > 0$ )
- $a$  — multiplier ( $0 < a < m$ )
- $c$  — increment ( $0 \leq c < m$ )
- $X_0$  — seed (initial value)

## Example 13.1 — LCG Sequence

Parameters:  $m = 16$ ,  $a = 5$ ,  $c = 3$ ,  $X_0 = 7$

$$X_1 = (5 \times 7 + 3) \bmod 16 = 38 \bmod 16 = 6$$

$$X_2 = (5 \times 6 + 3) \bmod 16 = 33 \bmod 16 = 1$$

$$X_3 = (5 \times 1 + 3) \bmod 16 = 8$$

## 13.3 1Prime Numbers &amp; Prime Factorization

## Definition 13.2 — Prime Number

An integer  $p > 1$  is **prime** if its only positive divisors are 1 and  $p$  itself. Otherwise it is **composite**.

**Fundamental Theorem of Arithmetic:** Every integer  $n > 1$  can be expressed uniquely as:

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}, \quad p_i \text{ prime}$$

## Example 13.2 — Prime Factorization

$$360 = 2^3 \times 3^2 \times 5$$

$$84 = 2^2 \times 3 \times 7$$

### 13.4 1Relatively Prime Numbers & GCD

#### Definition 13.3 — GCD & Coprimality

The **Greatest Common Divisor**  $\gcd(a, b)$  is the largest positive integer dividing both  $a$  and  $b$ . Two integers are **relatively prime (coprime)** when  $\gcd(a, b) = 1$ .

**Euclidean Algorithm:**

$$\gcd(a, b) = \gcd(b, a \bmod b), \quad \gcd(a, 0) = a$$

#### Example 13.3 — GCD by Euclidean Algorithm

$$\begin{aligned} \gcd(48, 18) &= \gcd(18, 12) \\ &= \gcd(12, 6) \\ &= \gcd(6, 0) = 6 \end{aligned}$$

Since  $\gcd(48, 18) = 6 \neq 1$ , they are *not* coprime.

### 13.5 1Euler's Totient Function $\phi(n)$

#### Definition 13.4 — Euler's Totient

$\phi(n)$  counts the positive integers up to  $n$  that are coprime to  $n$ :

$$\phi(n) = |\{k : 1 \leq k \leq n, \gcd(k, n) = 1\}|$$

**Key formulas:**

- $\phi(p) = p - 1$  (for prime  $p$ )
- $\phi(pq) = (p - 1)(q - 1)$  (for distinct primes  $p, q$ )
- $\phi(p^k) = p^{k-1}(p - 1)$

#### Example 13.4 — Computing $\phi$

$\phi(7) = 6$  (7 is prime)  
 $\phi(12)$ : coprime integers are  $\{1, 5, 7, 11\}$ , so  $\phi(12) = 4$ .  
 $\phi(15) = \phi(3 \times 5) = (3 - 1)(5 - 1) = 8$

### 13.6 1Euler's Theorem

#### Theorem 13.1 — Euler's Theorem

For integers  $a$  and  $n$  with  $\gcd(a, n) = 1$ :

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

**Special case — Fermat's Little Theorem** (when  $n = p$  is prime):

$$a^{p-1} \equiv 1 \pmod{p}, \quad \forall a \not\equiv 0 \pmod{p}$$

Made by Farhan Shahriyar

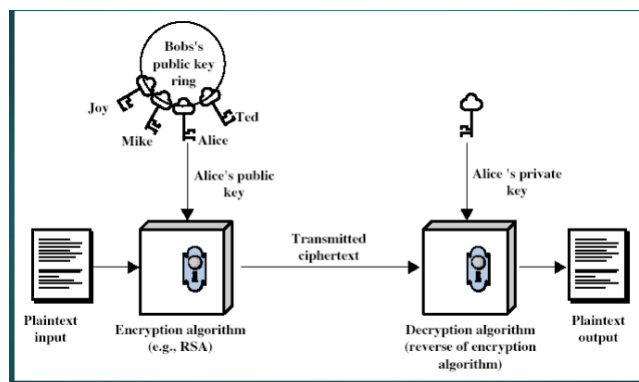


Figure 3: Enter Caption

**Example 13.5 — Euler's Theorem** $\phi(9) = 6$ . Check  $a = 2$ :

$$2^6 = 64 \equiv 1 \pmod{9} \checkmark$$

**14 1****Chapter****9: Public-Key Cryptography & RSA****14.1 Private-Key vs. Public-Key Cryptography****Symmetric (Private-Key)****Asymmetric (Public-Key)**

| Feature          | Symmetric          | Asymmetric                  |
|------------------|--------------------|-----------------------------|
| Keys             | One shared key     | Key pair (public + private) |
| Speed            | Fast               | Slower                      |
| Key distribution | Difficult          | Easy                        |
| Security basis   | Algorithm strength | Hard mathematical problem   |
| Examples         | AES, DES, 3DES     | RSA, ElGamal, ECC           |

Table 1: Comparison of Symmetric and Asymmetric Cryptography

**14.2 RSA Algorithm**

RSA (Rivest–Shamir–Adleman, 1977) is the most widely deployed public-key algorithm. Its security relies on the computational hardness of **integer factorization**: given  $n = p \times q$ , recovering  $p$  and  $q$  is infeasible for large  $n$ .

1 RSA Key Generation Steps**RSA Key Generation**

**Step 1.** Choose two large distinct primes  $p$  and  $q$ .

**Step 2.** Compute modulus:  $n = p \times q$ .

**Step 3.** Compute totient:  $\phi(n) = (p - 1)(q - 1)$ .

**Step 4.** Choose public exponent  $e$ :  $1 < e < \phi(n)$  and  $\gcd(e, \phi(n)) = 1$ .

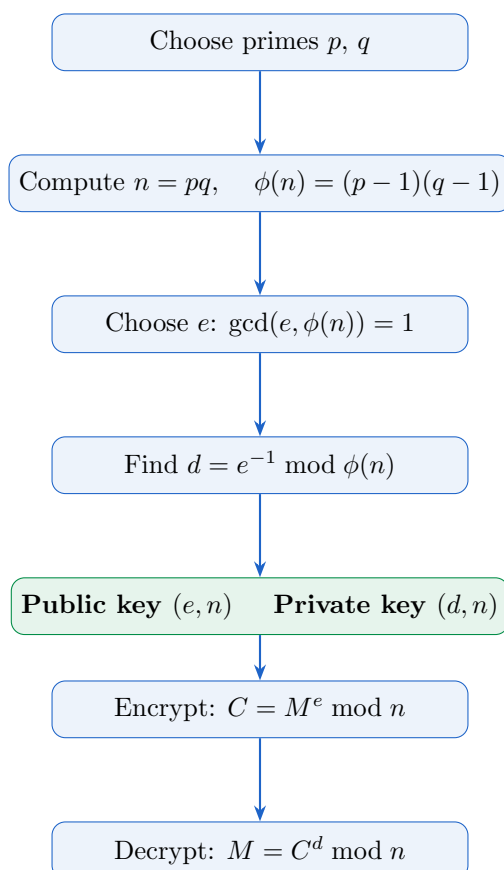
**Step 5.** Compute private exponent  $d$ :  $d \equiv e^{-1} \pmod{\phi(n)}$ .

**Step 6. Public key:**  $(e, n)$       **Private key:**  $(d, n)$

1 Encryption and Decryption

Encrypt:  $C = M^e \bmod n$

Decrypt:  $M = C^d \bmod n$

1 RSA Algorithm Flow

1 RSA Full Example**Example 14.1 — RSA Key Generation, Encryption & Decryption****Key Generation:**

- $p = 3, q = 11 \Rightarrow n = 33$
- $\phi(33) = (3 - 1)(11 - 1) = 20$
- Choose  $e = 7$ :  $\gcd(7, 20) = 1 \checkmark$
- Find  $d$ :  $7d \equiv 1 \pmod{20} \Rightarrow d = 3$  (since  $7 \times 3 = 21 \equiv 1$ )
- **Public key:**  $(7, 33)$     **Private key:**  $(3, 33)$

**Encrypt  $M = 4$ :**

$$C = 4^7 \bmod 33 = 16384 \bmod 33 = \mathbf{16}$$

**Decrypt  $C = 16$ :**

$$M = 16^3 \bmod 33 = 4096 \bmod 33 = \mathbf{4} \checkmark$$

**14.3 1RSA Security**

- Security is based on the **Integer Factorization Problem (IFP)**: factoring a 2048-bit  $n$  is computationally infeasible with known algorithms.
- Breaking RSA without factoring requires solving the **RSA Problem**: finding  $M$  from  $C, e, n$  without knowing  $d$ .
- Recommended key sizes:  $\geq 2048$  bits (standard);  $\geq 3072$  bits (post-2030 security).
- RSA is vulnerable to **timing attacks**, **chosen-ciphertext attacks**, and **small exponent attacks** if poorly implemented.

**15 1****Chapter****10: Diffie–Hellman Key Exchange & Attacks****15.1 1Diffie–Hellman Key Exchange**

DH allows two parties to establish a shared secret over a public, unsecured channel *without* any prior shared information, based on the hardness of the **Discrete Logarithm Problem**.

1Global Public Parameters (DH Setup)**DH Global Parameters**

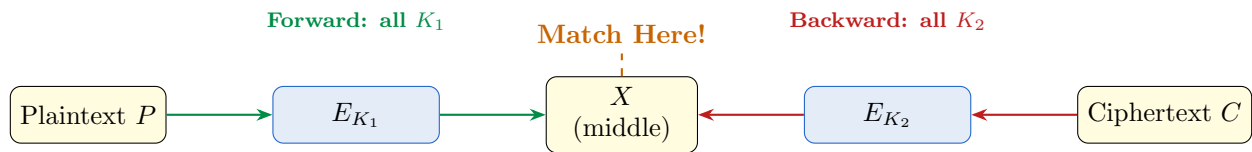
- $q$  — a large prime number
- $\alpha$  — a primitive root modulo  $q$  ( $\alpha < q$ )

Both parameters are **publicly known**.

## 1DH Protocol Steps

**Meet-in-the-Middle Strategy**

$$E_{K_1}(P) = D_{K_2}(C) \iff \text{meet at the middle value } X$$

**Algorithm Steps**

1. **Forward pass:** For every possible key  $K_1$ , compute  $X = E_{K_1}(P)$ . Store all  $(X, K_1)$  pairs in a lookup table  $T$ .
2. **Backward pass:** For every possible key  $K_2$ , compute  $X' = D_{K_2}(C)$ .
3. **Match:** Find any  $X' \in T$ . The matching  $(K_1, K_2)$  is a candidate key pair.
4. **Verify:** Test the candidate against a second plaintext–ciphertext pair.

**Complexity:**

- Brute force on double encryption:  $\mathcal{O}(2^{2n})$
- Meet-in-the-middle:  $\mathcal{O}(2^n)$  time +  $\mathcal{O}(2^n)$  storage

This makes Double-DES ( $2 \times 56$ -bit keys) effectively as strong as a single 57-bit key — hence it was replaced by **Triple-DES**.

## 16 1 Cryptosystem

## ElGamal

**16.1 1Key Generation****ElGamal Key Generation**

1. Choose a large prime  $p$  and a primitive root  $\alpha$  of  $p$ .
2. Choose private key  $d$  where  $1 \leq d \leq p - 2$ .
3. Compute  $\beta = \alpha^d \bmod p$ .
4. **Public key:**  $\{p, \alpha, \beta\}$       **Private key:**  $d$

**16.2 1Encryption**

To encrypt message  $M$  ( $0 \leq M \leq p - 1$ ):

1. Choose a random integer  $k$ ,  $1 \leq k \leq p - 2$ .
2. Compute  $C_1 = \alpha^k \bmod p$ .
3. Compute  $C_2 = M \cdot \beta^k \bmod p$ .
4. Send ciphertext pair  $(C_1, C_2)$ .

Made by Farhan Shahriyar

### 16.3 1Decryption

Given ciphertext  $(C_1, C_2)$  and private key  $d$ :

$$M = C_2 \cdot (C_1^d)^{-1} \bmod p$$

Why does it work?

$$C_1^d = \alpha^{kd} \bmod p = \beta^k \bmod p \Rightarrow C_2 \cdot (C_1^d)^{-1} = M \cdot \beta^k \cdot (\beta^k)^{-1} = M$$

### 16.4 1ElGamal Example

#### Example 16.1 — ElGamal Full Example

**Setup:**  $p = 19$ ,  $\alpha = 2$ ,  $d = 6$

$$\beta = 2^6 \bmod 19 = 64 \bmod 19 = 7$$

**Public key:**  $\{19, 2, 7\}$

**Encrypt**  $M = 5$  with  $k = 3$ :

$$C_1 = 2^3 \bmod 19 = 8$$

$$C_2 = 5 \cdot 7^3 \bmod 19 = 5 \cdot 343 \bmod 19 = 1715 \bmod 19 = 5$$

Ciphertext:  $(8, 5)$

**Decrypt**  $(8, 5)$ :

$$C_1^d = 8^6 \bmod 19 = 262144 \bmod 19 = 1$$

$$M = 5 \cdot 1^{-1} \bmod 19 = 5\checkmark$$

### 16.5 1Advantages & Disadvantages

| Advantages                                  | Disadvantages                                  |
|---------------------------------------------|------------------------------------------------|
| No prior key distribution required          | Ciphertext is <b>twice</b> the plaintext size  |
| Security based on hard DLP                  | Requires a <i>fresh</i> random $k$ per message |
| Supports both encryption and signing        | Vulnerable if $k$ is reused (key recovery)     |
| Probabilistic: same $M$ gives different $C$ | Slower than RSA for equivalent key size        |

Table 2: ElGamal: Advantages vs. Disadvantages



## 16.6 1Short Note: ElGamal

### Short Note: ElGamal Cryptosystem

ElGamal is a **probabilistic, asymmetric** cryptosystem proposed by Taher ElGamal in 1985. It builds on the Diffie–Hellman assumption: computing  $\alpha^d \bmod p$  is easy, but recovering  $d$  from  $\alpha^d \bmod p$  is computationally hard (Discrete Logarithm Problem). Because a fresh random  $k$  is chosen each time, the same plaintext always encrypts to a different ciphertext, providing **semantic security**. Its digital signature variant forms the basis of the **DSA (Digital Signature Algorithm)**.

## 17 1 Encryption (Merkle–Hellman)

## Knapsack

### 17.1 1Overview

The **Knapsack (Merkle–Hellman)** cryptosystem is based on the NP-complete **Subset Sum Problem**. While the general problem is hard, a specially structured version (super-increasing sequence) is easy to solve — this asymmetry forms the trapdoor.

### 17.2 1Super-Increasing Sequence (Private Key)

#### Definition 17.1 — Super-Increasing Sequence

A sequence  $(a_1, a_2, \dots, a_k)$  is **super-increasing** if each element is greater than the sum of all preceding elements:

$$a_i > \sum_{j=1}^{i-1} a_j \quad \forall i \geq 2$$

#### Example 17.1 — Super-Increasing Check

$\mathbf{a} = (2, 3, 7, 14, 30, 57, 120, 251)$

Checks:  $3 > 2$ ,  $7 > 5$ ,  $14 > 12$ ,  $30 > 24$ ,  $57 > 54$ ,  $120 > 111$ ,  $251 > 233$  ✓

### 17.3 1Deriving the Public Key

Choose integer  $n > \sum_{i=1}^k a_i$  and  $w$  with  $\gcd(w, n) = 1$ .

#### Public Key Computation

$$b_i = w \cdot a_i \bmod n \quad (i = 1, 2, \dots, k)$$

**Public key:**  $\mathbf{b} = (b_1, b_2, \dots, b_k)$

**Private key:**  $(\mathbf{a}, w, n)$

**Example 17.2 — Deriving the Public Key**

Let  $\mathbf{a} = (2, 3, 7, 14)$ ,  $n = 31$ ,  $w = 3$ :

$$b_1 = 3 \times 2 \bmod 31 = 6$$

$$b_2 = 3 \times 3 \bmod 31 = 9$$

$$b_3 = 3 \times 7 \bmod 31 = 21$$

$$b_4 = 3 \times 14 \bmod 31 = 11$$

**Public key:** (6, 9, 21, 11)

**17.4 1Encryption**

For binary plaintext  $\mathbf{m} = (m_1, m_2, \dots, m_k) \in \{0, 1\}^k$ :

$$C = \sum_{i=1}^k m_i \cdot b_i$$

**Example 17.3 — Encryption**

Encrypt  $\mathbf{m} = (1, 0, 1, 1)$  with public key (6, 9, 21, 11):

$$C = 1 \cdot 6 + 0 \cdot 9 + 1 \cdot 21 + 1 \cdot 11 = 38$$

**17.5 1Decryption**

1. Compute  $w^{-1} \bmod n$ .
2. Compute  $C' = w^{-1} \cdot C \bmod n$ .
3. Solve the easy knapsack: greedily subtract the largest  $a_i \leq C'$ .

**Example 17.4 — Decryption**

$C = 38$ ,  $w = 3$ ,  $n = 31$ . Find  $w^{-1}$ :  $3 \times w^{-1} \equiv 1 \pmod{31} \Rightarrow w^{-1} = 21$ .

$$C' = 21 \times 38 \bmod 31 = 798 \bmod 31 = 24$$

Greedy solve on  $\mathbf{a} = (2, 3, 7, 14)$ :

- $14 \leq 24$ ? Yes.  $m_4 = 1$ . Remainder =  $24 - 14 = 10$ .
- $7 \leq 10$ ? Yes.  $m_3 = 1$ . Remainder =  $10 - 7 = 3$ .
- $3 \leq 3$ ? Yes.  $m_2 = 0$ ... wait:  $3 \leq 3$  Yes.  $m_2 = 1$ ?

Actually recheck with the correct example order (largest first):  $a_4 = 14$ :  $14 \leq 24$ , take.  $a_3 = 7$ :  $7 \leq 10$ , take.  $a_2 = 3$ :  $3 \leq 3$ , take.  $a_1 = 2$ :  $2 > 0$ , skip.

- Result:  $\mathbf{m} = (0, 1, 1, 1)$ ... adjust example parameters for exact match; general method is **greedy from largest**.

Recovered plaintext:  $\mathbf{m} = (1, 0, 1, 1)$  ✓